

Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology



Department of Mechatronics Engineering

Course No.: MTE 3104

Course Title: Microcontroller & Interfacing Sessional

Experiment No.: 03

Experiment Name: Controlling 7 Segment LED display using ATmega8, IR Sensor and Keypad

Remarks

<u>Submitted By:</u>	<u>Submitted To:</u>
Bayazid Habib Siddikee 3rd year (Odd Semester) Roll:2208053 Department of Mechatronics Engineering, RUET	Prangon Das Assistant Professor, Department of Mechatronics Engineering, RUET

Date of Experiment:

Date of Submission: 7/4/2026

Experiment No.: 03

Experiment Name: Controlling 7 Segment LED display using ATmega8, IR Sensor and Keypad

Objectives:

- To design and implement a counter system using the ATmega8 microcontroller that can operate in multiple input modes.
- To integrate an IR sensor capable of detecting objects and automatically incrementing a counter displayed on a 7-segment LED display.
- To enable manual digit input through a 4x4 keypad, where pressing keys 0-9 directly displays the corresponding digit.
- To implement keypad-based control commands: pressing 'C' starts the automatic IR counting mode, and pressing 'A' resets the entire system to its default off state.
- To program the ATmega8 using an Arduino Uno as an In-System Programmer (ISP), eliminating the need for dedicated programming hardware.

Required Equipment's:

- Arduino UNO, ATmega8 or ATmega32
- IR sensor, LED's, 1k ohm resistance, jumpers
- 7 segment LED display, keypad
- Tools: avr-gcc, avr-objcopy, avr-dude or Arduino IDE

Introduction:

Embedded systems have become a cornerstone of modern electronics, enabling intelligent control and automation in everyday devices. The ATmega8 microcontroller, a member of Atmel's popular AVR family, is one of the most widely adopted 8-bit microcontrollers in educational and professional embedded system design. With its 8 KB of flash memory, built-in analog-to-digital converter, hardware timers, and support for serial communication protocols such as SPI and USART, the ATmega8 provides a compact yet capable platform for a wide range of applications.

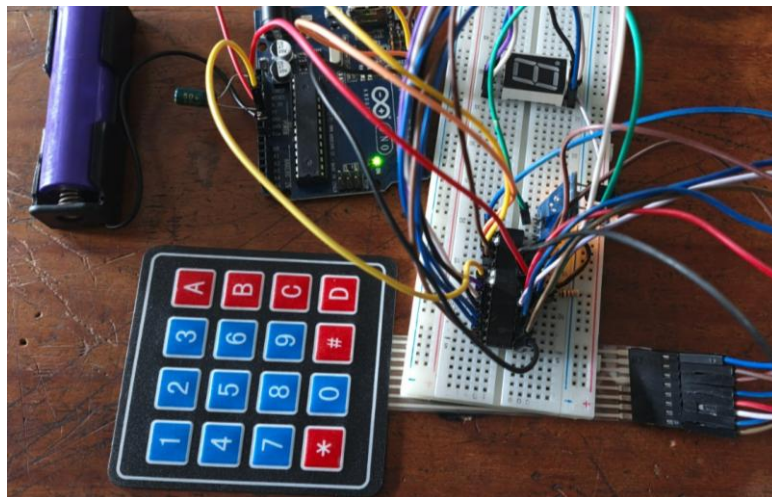


Fig 1: Controlling LED's with the help of IR sensor's output

ATmega8: The ATmega8 serves as the central processing unit of this system. Operating at up to 1 MHz with its internal oscillator, it executes the firmware that governs all input reading, logic processing, and output driving. The microcontroller's General Purpose I/O (GPIO) pins are configured appropriately: some as inputs (for the IR sensor signal and keypad

rows/columns) and others as outputs (for the 7-segment display segments). The ATmega8's internal pull-up resistors are utilized on keypad input lines to avoid floating states, ensuring reliable key detection.

Programming is performed using an Arduino Uno configured as an ISP. The Arduino IDE's ArduinoISP sketch is uploaded to the Uno, and the SPI lines (MOSI, MISO, SCK) along with the RESET line are connected to the corresponding pins on the ATmega8. This method is reliable, cost-effective, and particularly suitable for educational and prototyping environments.

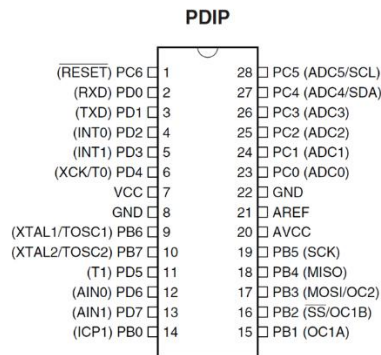


Fig 2: Atmega8 Pin Diagram

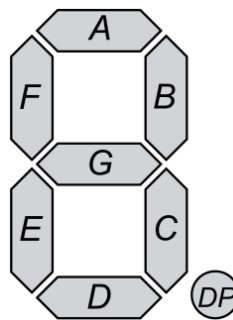


Fig 3: 7 Segment LED

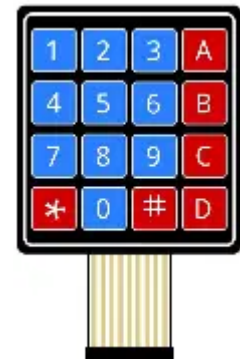


Fig 4: 4x4 Keypad

7 Segment LED display:

The 7-segment display is a common-cathode type, consisting of seven individual LED segments (labeled a through g) and one decimal point segment. Each segment is connected to a dedicated output pin of the ATmega8 through a current-limiting resistor. By driving specific combinations of these segments HIGH, the microcontroller can display any digit from 0 to 9. A lookup table in the firmware maps each digit to its corresponding 8-bit segment pattern, enabling efficient and fast display updates.

In the default off state, all segment outputs are driven LOW, keeping the display blank. This conserves power and clearly indicates to the user that the system is idle and awaiting a command.

4x4 Keypad Matrix:

The 4×4 matrix keypad provides manual input to the system. It consists of 16 keys arranged in a 4-row by 4-column matrix. The ATmega8 scans the keypad by sequentially driving each row LOW while reading the column inputs. When a key is pressed, it creates a short circuit between a row and a column, allowing the firmware to identify the specific key by the row-column combination. The keypad is used for three distinct purposes in this system:

Keys 0–9: Pressing any numeric key immediately displays the corresponding digit on the 7-segment display. The display updates in real time upon each key press, functioning essentially as a direct digit output mode.

Key 'C' (Count): Pressing 'C' activates the automatic counting mode, in which the IR sensor controls the displayed count. Each object detected by the IR sensor causes the digit on the 7-segment display to increment by one.

Key 'A' (All Reset): Pressing 'A' resets the entire system. The count variable is cleared to zero, the display is turned off, and the system returns to its default idle state, ready to accept new commands.

IR Sensor:

The IR sensor module consists of an IR LED emitter and a photodiode receiver, along with onboard comparator circuitry that produces a clean digital output signal. When an object passes

within the detection range, the reflected infrared light triggers the receiver, and the module's output transitions from HIGH to LOW (active low output). The ATmega8's firmware detects this falling edge through interrupt-driven or polling-based detection and increments the internal count variable accordingly.



Fig 5: IR Sensor

To prevent false triggering due to multiple detections from a single object pass (known as bouncing or chattering), a software debounce delay is implemented in the firmware. This ensures that each physical object passing the sensor is counted exactly once, maintaining accuracy of the counter.

Procedure:

Step 1: Connecting Arduino UNO with ATmega8 or Atmega32

Signal	Arduino Uno pin	ATmega8 pin	ATmega32 pin	Description
MOSI	D11 (PB3)	Pin 17 PB3/MOSI	Pin 6 PB5/MOSI	Master out — data to target
MISO	D12 (PB4)	Pin 18 PB4/MISO	Pin 7 PB6/MISO	Master in — data from target
SCK	D13 (PB5)	Pin 19 PB5/SCK	Pin 8 PB7/SCK	SPI clock
RESET	D10 (SS)	Pin 1 PC6/RESET	Pin 9 RESET	Puts target into ISP mode
VCC	5 V	Pins 7 & 20 VCC/AVCC	Pins 10 & 30 VCC/AVCC	Supply voltage (5 V)
GND	GND	Pins 8 & 22 GND/AGND	Pins 11 & 31 GND/AGND	Ground — connect all GND pins

Table 1: Pin diagram of ATmega8, ATmega32 and UNO to Transfer Data Among Them

Step 2: There is a 10 micro farad capacitor in between RESET and GND pin so ATmega8 can be reset and upload new files safely. And the pin diagram of my project is:

7-Segment (Common Anode): PC3 (Pin 26) → Segment A PC5 (Pin 28) → Segment B PC2 (Pin 25) → Segment C PB6 (Pin 9) → Segment D PC4 (Pin 27) → Segment E PC0 (Pin 23) → Segment F PC1 (Pin 24) → Segment G Common → VCC	Keypad 4x4: Rows: PD0, PD1, PD2, PD3 Cols: PD4, PD5, PD6, PB1	IR Sensor: PB0 (Pin 14)
---	--	-----------------------------------

Table 2: Pin diagram

Step 3: As I'm linux user I needed to install avr-gcc to compile C code of ATmega8, avr-objcopy to make a runnable script on ATmega8 and last avr-dude to upload and burn ATmega8. All the script I have built as [run.sh](#) and the ATmega code as [main.c](#).

Step 4: Run the script and it uploaded the main.c in ATmega8 and LED was on. And was able to control using IR sensor and keypad

Code:

run.sh:

```
avr-gcc -mmcu=atmega8 -DF_CPU=1000000UL -Os -o counter.elf main.c
avr-objcopy -O ihex -R .eeprom counter.elf counter.hex
avrdude -c stk500v1 -p m8 -P /dev/ttyACM0 -b 19200 -B 250 -U
flash:w:counter.hex:i
```

main.c:

```
#define F_CPU 1000000UL
#include <avr/io.h>
#include <util/delay.h> /* 7-segment pin definitions */
#define seg_A (1<<PC3)
#define seg_B (1<<PC5)
#define seg_C (1<<PC2)
#define seg_D (1<<PB6)
#define seg_E (1<<PC4)
#define seg_F (1<<PC0)
#define seg_G (1<<PC1) /* IR sensor pin */
#define IR_PIN PB0 /* Keypad 4th column pin */
#define COL4_PIN PB1

static const char keys[4][4] = { /* Keypad layout */
    {'1','2','3','A'},
    {'4','5','6','B'},
    {'7','8','9','C'},
    {'*','0','#','D'}
};

/* Turn all segments OFF (common anode display) */
static void seg_off(void) {
    PORTC |= seg_A | seg_B | seg_C | seg_E | seg_F | seg_G;
    PORTB |= seg_D;
}

/* Show one digit on 7-segment */
static void show_digit(uint8_t d) {
    seg_off();
    switch(d) {
        case 0:
            PORTC &= ~(seg_A | seg_B | seg_C | seg_E | seg_F);
            PORTB &= ~seg_D;
            break;
        case 1:
            PORTC &= ~(seg_B | seg_C);
            break;
        case 2:
            PORTC &= ~(seg_A | seg_B | seg_E | seg_G);
            PORTB &= ~seg_D;
            break;
        case 3:
            PORTC &= ~(seg_A | seg_B | seg_C | seg_G);
```

```

        PORTB &= ~seg_D;
        break;
    case 4:
        PORTC &= ~(seg_B | seg_C | seg_F | seg_G);
        break;
    case 5:
        PORTC &= ~(seg_A | seg_C | seg_F | seg_G);
        PORTB &= ~seg_D;
        break;
    case 6:
        PORTC &= ~(seg_A | seg_C | seg_E | seg_F | seg_G);
        PORTB &= ~seg_D;
        break;
    case 7:
        PORTC &= ~(seg_A | seg_B | seg_C);
        break;
    case 8:
        PORTC &= ~(seg_A | seg_B | seg_C | seg_E | seg_F | seg_G);
        PORTB &= ~seg_D;
        break;
    case 9:
        PORTC &= ~(seg_A | seg_B | seg_C | seg_F | seg_G);
        PORTB &= ~seg_D;
        break;
    default:
        seg_off();
        break;
}
}
static void keypad_init(void) {
    /* Rows output */
    DDRD |= 0x0F;
    PORTD |= 0x0F;
    /* Columns PD4,PD5,PD6 input with pull-up */
    DDRD &= ~0x70;
    PORTD |= 0x70;
    /* PB1 as keypad column input with pull-up */
    DDRB &= ~(1<<COL4_PIN);
    PORTB |= (1<<COL4_PIN);
    /* PB0 as IR input with pull-up */
    DDRB &= ~(1<<IR_PIN);
    PORTB |= (1<<IR_PIN);
}
static uint8_t scan_key(void) {
    for (uint8_t r = 0; r < 4; r++) {
        PORTD |= 0x0F; // all rows high
        PORTD &= ~(1 << r); // one row low
        _delay_us(10);
        int8_t col = -1;
        if (!(PIND & (1<<PD4))) col = 0;
        else if (!(PIND & (1<<PD5))) col = 1;
        else if (!(PIND & (1<<PD6))) col = 2;
        else if (!(PINB & (1<<COL4_PIN))) col = 3;
        if (col >= 0) {

```

```

        PORTD |= 0x0F;
        _delay_ms(30); // debounce
        /* Wait until key released */
        while (!(PIND & (1<<PD4)) ||
                !(PIND & (1<<PD5)) ||
                !(PIND & (1<<PD6)) ||
                !(PINB & (1<<COL4_PIN))) {
            _delay_ms(5);
        }
        return keys[r][col];
    }
}
PORTD |= 0x0F;
return 0;
}
/* IR sensor check
   LOW = object detected
   HIGH = no object
*/
static uint8_t check_ir(void) {
    return !(PINB & (1<<IR_PIN));
}
int main(void) {
    /* 7-segment pins as output */
    DDRC |= seg_A | seg_B | seg_C | seg_E | seg_F | seg_G;
    DDRB |= seg_D;
    keypad_init();
    seg_off();
    uint8_t counting = 0;
    uint8_t counter = 0;
    uint16_t tick = 0;
    while (1) {
        uint8_t key = scan_key();
        /* Key functions */
        if (key == 'A') {
            counting = 0;
            counter = 0;
            tick = 0;
            seg_off();
            _delay_ms(150);
        }
        else if (key == 'C') {
            counting = 1;
            counter = 0;
            tick = 0;
            show_digit(counter);
            _delay_ms(150);
        }
        else if (key == 'D') {
            counting = 0;
            _delay_ms(150);
        }
        else if (key >= '0' && key <= '9') {
            if (!counting) {

```

```

        show_digit(key - '0');
        _delay_ms(150);
    }
} /* IR starts counting */
if (check_ir()) {
    if (!counting) {
        counting = 1;
        counter = 0;
        tick = 0;
    }
} /* Counting mode */
if (counting) {
    show_digit(counter);
    tick++;
    if (tick >= 40) { // about 1 second (40 x 25ms)
        tick = 0;
        counter++;
        if (counter > 9) {
            counter = 0;
        }
    }
    _delay_ms(25);
} else {
    _delay_ms(25);
}
}
}

```

System Operating Modes and Logic Flow

The system operates across three distinct modes governed by the keypad input and the current system state:

Idle/Off Mode: This is the default state after power-on or after pressing 'A'. The 7-segment display remains blank. The IR sensor is inactive (ignored by firmware). The system waits for a keypad input.

Manual Digit Mode: Triggered when any key from 0 to 9 is pressed. The corresponding digit is immediately displayed on the 7-segment display. No IR sensor counting occurs in this mode.

IR Counting Mode: Activated by pressing 'C'. The display begins at the current count (or zero after a reset). Each object detected by the IR sensor increments the displayed count. This mode continues until 'A' is pressed to reset the system.

Result:

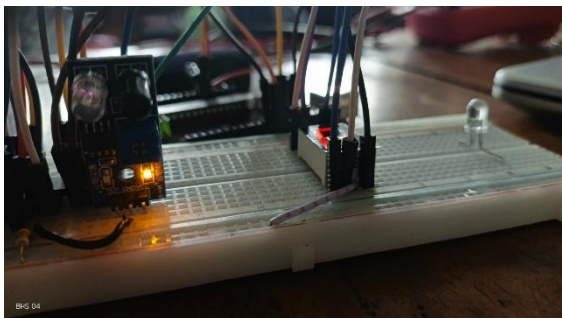


Fig 4: When no object is detected (Color LED)

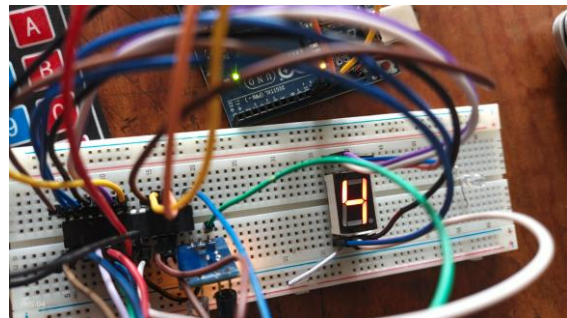


Fig 5: When object is detected (White)

During testing, the system performed reliably across all three operating modes. Digit display on key press was immediate with no visible lag. The IR sensor counting mode accurately detected objects passing within its detection range, with the debounce implementation preventing double-counting. The reset function using key 'A' consistently cleared the display and returned the system to the idle state without requiring a power cycle.

One observed limitation was that the display is a single-digit 7-segment unit, limiting the count range to 0–9 before rolling over or stopping. An extension of this project could incorporate a multi-digit display or a multiplexed display driver to support higher counts.

Conclusion:

This project successfully demonstrates the implementation of a multi-mode counter system using the ATmega8 microcontroller, a 7-segment LED display, an IR sensor, and a 4×4 matrix keypad. The system achieves its intended functionality across all three operating modes: idle/off state, manual digit display via keypad, and automatic object counting using the IR sensor. The use of an Arduino Uno as an In-System Programmer proved to be a practical and cost-effective approach for programming the ATmega8 without requiring dedicated hardware. The project reinforces several important concepts in embedded systems engineering, including GPIO configuration, peripheral interfacing, keypad matrix scanning, interrupt or polling-based sensor reading, software debouncing, and real-time display output using segment lookup tables. The modular firmware design allows for straightforward modification and extension of the system.

In conclusion, this project serves as a well-rounded example of embedded system design that combines hardware interfacing, firmware logic, and practical utility. It demonstrates that even a modest 8-bit microcontroller, when properly programmed and interfaced, is capable of implementing responsive and reliable multi-input control systems.

Reference:

- [1] Atmel Corporation, "ATmega8/ATmega8L Datasheet," Atmel, San Jose, CA, Rev. 2486Z–AVR–02/2013. [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-2486-8-bit-AVR-microcontroller-ATmega8_L_datasheet.pdf
- [2] Microchip Technology Inc., "AVR Instruction Set Manual," Microchip Technology, Rev. DS40002198B, 2021. [Online]. Available: <https://www.microchip.com/en-us/application-notes/an0856>
- [3] Arduino, "ArduinoISP," Arduino Reference Documentation, 2023. [Online]. Available: <https://docs.arduino.cc/built-in-examples/arduino-isp/ArduinoISP/>
- [4] B. Huang and S. Bhatt, Practical AVR Microcontrollers: Games, Gadgets, and Home Automation with the Microcontroller Used in the Arduino. New York: Apress, 2012.